Ultron Memory System: Engineering Design Report

Project owner: Nitin Kumar Yadav · 5 November 2026 · Status: design proposal for engineering review, no code written yet

1. Summary

Ultron must remember every conversation, learn from it, and use that memory in real time without slowing replies. This report proposes a custom memory layer on SQLite, stored locally and never committed to GitHub. It combines a permanent raw log with extracted facts, learned workflows and ingested sources (WhatsApp, Gmail, YouTube, personal files). Growth comes from retrieval and background consolidation, not model training. Every line is kept so the model can be fine-tuned later.

Decisions already made by the project owner:

- SQLite is the memory store.
- No model training for now. Ultron gets a personality prompt and adapts from past conversations.
- Every typed and spoken line is saved and kept for future training.
- Memory lives in a separate local location on the laptop, never in the git repository.
- Ultron must learn taught workflows (for example, “go to Claude, type this, open this project”) and material from WhatsApp, Gmail, YouTube and personal files.
- Reply speed must not get worse.

2. Context and problem

- **Platform:** Fedora Linux laptop. LFM3-1.7B (GGUF, Q6_K) served by llama.cpp’s llama-server. Voice pipeline in progress. The owner is building Ultron’s own environment and dashboard.
- **Hardware constraint:** 4 GB VRAM (GTX 1650) shared with speech models. System RAM must be confirmed before sizing background jobs.
- **Problem:** Ultron has no persistence, converses and handles commands poorly, and starts every session blank.
- **Design consequence:** a 1.7B model reasons weakly, so memory must hand it small, precise, relevant context, and routine commands should bypass the model entirely.

3. Requirements

ID	Requirement
R1	Log every typed and spoken line, plus Ultron's replies, actions and outcomes, permanently and append-only
R2	Recall relevant memory within a session and across sessions
R3	Learn facts, preferences and corrections from conversations without retraining
R4	Learn taught workflows and replay them on request
R5	Ingest WhatsApp chats, Gmail, YouTube videos and personal files as searchable knowledge
R6	Memory overhead on the response path of about 50 ms at p95 (design target, to be measured)
R7	All data local; nothing in git; no cloud calls for memory
R8	Every stored item traceable to its source; the owner can review, edit and hide items
R9	The full log can later be exported as training data

4. Research findings

4.1 Memory types

Surveys split agent memory into working (the current prompt), episodic (timestamped events), semantic (facts) and procedural (skills and workflows). The hard part is consolidation: turning raw episodes into facts and skills.

4.2 How agents grow without retraining

- **Reflexion:** after a failure the agent writes a short lesson and reads it next time.

- **Voyager skill library:** verified successful action programs are stored and retrieved for similar tasks.
- **Agent Workflow Memory (Wang et al., 2024):** induces reusable workflows from past successful runs and feeds them back to the agent. The paper reports relative success gains of 24.6% on Mind2Web and 51.1% on WebArena, with fewer steps.
- **Letta sleep-time compute:** a background process reorganises memory between conversations, off the critical path.
- **Zep / Graphiti:** facts carry validity windows. A contradicted fact is invalidated, not deleted.
- **Experience replay:** similar past command and action pairs are injected into the prompt as few-shot examples. Cheap and well suited to a small model.

4.3 Projects evaluated

Project	What it offers	Verdict
Mem0 (Apache 2.0)	LLM-extracted memories with search; vendor-reported benchmark gains	Extraction needs reliable structured output, which small models struggle with. Revisit if a larger model runs during idle time.
Letta (Apache 2.0)	Always-in-context core blocks, searchable history, archival store, sleep-time consolidation	Adopt the ideas, not the runtime. Self-editing memory depends on model judgment.
Zep / Graphiti	Temporal knowledge graph with fact validity windows	Adopt invalidate-not-delete. The graph database and per-episode LLM extraction are too heavy.
Cognee	Knowledge-graph memory engine with a read-only, incremental Gmail sync using Gmail's history cursor	Use its Gmail sync design as a reference.
Obsidian vaults	Human-readable markdown notes	Export and audit only. Too slow and unstructured as the live store.

Project	What it offers	Verdict
Lightweight SQLite projects (MemCore, Ariadne, TinyContext, vstash)	FTS5 keyword plus sqlite-vec vector search, rank fusion, token-budget trimming	Closest to our pattern. These are young projects, so treat them as reference designs.

Verdict: build a small custom layer. It stays fast on a 1.7B model, and the habit, workflow and correction tables Ultron needs do not exist off the shelf.

5. Architecture

```
Input (voice / text / dashboard)
  -> Voice gate (VAD, speaker check)
  -> Router: exact alias or workflow match? yes -> run action, skip the LLM
  -> Prompt builder: static prefix + core block + retrieved memory + recent
turns
  -> llama-server (LFM3-1.7B, constrained output)
  -> Action executor (confirmation policy for risky actions)
  -> Reply (voice / text)
  -> Event logger -> write queue -> SQLite (single writer)

Background workers (idle only)
  Consolidation: facts, habits, workflows, summaries, decay
  Ingestion: WhatsApp exports, Gmail sync, YouTube transcripts, files
  Embeddings (later phase)
```

Process model: one memory service with a single writer thread and a queue (SQLite allows one writer at a time), read connections per thread, WAL mode. An in-RAM cache holds the core block, aliases and active workflows and is refreshed on every write. The dashboard uses a read-only connection and writes through the service.

6. Data model

```
PRAGMA journal_mode=WAL; PRAGMA synchronous=NORMAL; PRAGMA foreign_keys=ON;

sessions(id, started_at, ended_at, channel)
events(id, session_id, ts, speaker, channel, text, asr_conf, speaker_score,
       intent, action_json, outcome, latency_ms, retrieved_ids, hidden,
source_id)
facts(id, key, value, kind, confidence, valid_from, valid_to, user_verified,
      source_event_ids, last_used, use_count)
workflows(id, name, trigger_phrases, steps_json, risk_level, successes,
failures,
          last_success, created_by, user_verified, trusted, active)
corrections(id, event_id, heard, did, wanted, ts)
summaries(id, scope, period_start, period_end, text, source_event_ids)
sources(id, kind, uri_or_path, title, content_hash, ingested_at, trust,
enabled)
chunks(id, source_id, ord, text, token_count)
metrics(id, ts, stage, ms, event_id)
-- FTS5 indexes over events.text, facts.value, chunks.text
-- later: sqlite-vec table of embeddings for facts, summaries and chunks
```

Principles:

- **Append-only:** events are never updated or deleted. Hiding uses the hidden flag and removes the row from retrieval and learning only.
- **Provenance:** every fact, workflow and summary stores the IDs of the events or sources it came from.
- **Time:** UTC timestamps. A changed fact closes the old row with valid_to and opens a new one.
- **Trust:** sources.trust marks external content as data, never instructions.
- **Audio:** raw audio is not stored by default (open decision, see section 17).

7. Read path (real-time recall)

1. **Core block** in RAM: about ten lines covering the owner’s setup, current project and active context. Rebuilt only when facts change, so the prompt prefix stays stable.
2. **Exact lookup** of aliases and taught workflows, from RAM or indexed SQL. A hit runs the action with no LLM call.

3. **Keyword search** (FTS5) over facts, recent events and ingested chunks. Top few results only.
4. **Similar past successes:** one to three command and action pairs as few-shot examples.
5. **Vector search** only when steps 2 to 4 find nothing (Phase 5).

Cap retrieved memory in count and tokens (starting point: 300 to 500 tokens, tune by measurement). External snippets are wrapped in delimiters with their source and labelled as data.

Prompt layout for caching. Order the prompt as: static persona, rules and tool schema, then the core block, then retrieved memory, then recent turns, then the new input. Liquid's llama.cpp guide says llama-server keeps the previous request's KV cache and processes only the new suffix (cache_prompt is on by default), and advises keeping long system prompts and retrieval preambles byte-identical. Its --cache-reuse option (the guide uses 256) reuses chunks by shifting when an earlier part changed. A llama.cpp discussion warns that a timestamp or random ID early in the prompt breaks the cache. So no timestamps or IDs in the prefix; time and session details go at the end.

8. Write path

- Each event goes onto an in-memory queue and is written by the single writer after the reply has started, so the owner never waits on a disk write.
- WAL with synchronous=NORMAL is the proposed default. A power cut can lose the last few transactions but does not corrupt the file.
- Flush the queue on shutdown. Run periodic WAL checkpoints and a nightly copy with SQLite's online backup to a separate folder. Run an integrity check at startup.
- Taught instructions and corrections write through to the RAM cache immediately, so they apply on the very next turn.

9. Learning loop (how Ultron grows)

In the session (real time): corrections, "remember this" commands and session state take effect on the next turn through the RAM cache.

While idle (consolidation worker, low priority, runs only after a quiet period):

1. Read new events since the last checkpoint.

2. Extract candidate facts, preferences and workflows with constrained JSON output (llama.cpp supports grammar and JSON-schema constraints; confirm the option for the chosen build), small schemas and rule-based validation. If GPU memory is free, a larger model can run here only.
3. Resolve conflicts by closing the old fact's valid_to. New facts below a confidence threshold wait for owner confirmation.
4. Promote habits: a command-to-action mapping becomes a habit after repeated verified successes (start at 3, configurable). Corrections outrank everything and are stored as heard, did, wanted triples.
5. Induce workflows from repeated multi-step sequences (Agent Workflow Memory style), stored as drafts for approval in the dashboard.
6. Write session and daily summaries. Lower the ranking of unused facts over time; never delete them.
7. Never learn from failed or misheard turns or from external content as instructions, and never auto-learn destructive actions.

10. Teaching Ultron

10.1 Taught workflows

The owner says something like: "remember, when I say start work on Ultron, open Claude, type this prompt, and open the Ultron project folder." This is stored immediately as a workflow with trigger phrases, ordered steps, variables (for example the project name), a risk level and user_verified set. The action executor replays it; the LLM only maps the owner's phrase to a workflow ID. Steps that type into apps, send messages, delete or pay require confirmation until the owner marks the workflow trusted.

10.2 Sources

Source	Method	Notes
WhatsApp	Owner exports a chat (Export chat, without media) to a .txt file in a watched folder; a parser ingests it	Personal accounts have no official read API. Unofficial libraries (whatsapp-web.js, Baileys, whatsmeow) violate WhatsApp's terms and risk account bans. The official Cloud API targets business accounts. Exports omit deleted messages and replace media with a placeholder.
Gmail	Official Gmail API, OAuth desktop-app client, read-only scope (gmail.readonly), incremental sync with Gmail's history cursor	Restricted scope. Test-user mode works without Google verification, but one source reports testing-mode tokens expire after 7 days. Confirm current Google policy before building. Sync only chosen labels.
YouTube	Transcript first: youtube-transcript-api or yt-dlp subtitles, preferring manual captions. Fallback: download audio and transcribe with local Whisper after owner approval	Store transcript, chapters if available, and a summary. Only videos the owner selects.
Personal files	Watched folder for txt, md, pdf and docx; optional Obsidian vault import	Dedupe by content hash; re-ingest on change.

10.3 Ingestion pipeline

Parse, normalise, dedupe by content hash, chunk (starting point 300 to 500 tokens with small overlap), add to the FTS index. Idle jobs add embeddings, summaries and facts, always with source IDs. Each source has an on/off switch, an allow list (chats, labels, channels) and a one-click purge.

10.4 Learning from what is on screen

The owner wants to say “we are in this WhatsApp chat, learn it.” First version: the voice command triggers the export or sync flows above. True screen reading (accessibility tree or OCR) is a separate research item that has not been investigated.

11. Latency design

- Fast path skips the LLM on exact alias and workflow matches.
- RAM cache for the core block, aliases and workflows.
- Stable prompt prefix and KV cache reuse (section 7).
- Retrieval capped in count and tokens, with no LLM calls inside retrieval.
- Writes and all learning off the hot path.
- Warm start: on boot load the caches and send a priming request with the static prefix.
- Per-stage timings go into the metrics table, and the dashboard shows p50 and p95 so regressions are visible.

Targets: owner goal of 2 to 3 seconds end to end; memory overhead about 50 ms at p95 (design target, to be measured).

12. Privacy, security and safety

- **Location:** ~/.local/share/ultron/ outside the repository, folder permissions limited to the owner. Add *.db, *.db-wal and *.db-shm to .gitignore and add a pre-commit check that fails if any is staged.
- **No cloud:** if a cloud model is ever used, memory snippets in the prompt would leave the laptop. Keep memory features local-only by default.
- **Encryption:** recommend full-disk encryption (LUKS) since the log holds a great deal of personal data. SQLCipher is an option for file-level encryption.
- **Secrets:** filter passwords, card and ID numbers and API keys before logging.
- **Third-party data:** WhatsApp and Gmail contain other people’s messages. Ingest only what the owner selects.
- **Memory poisoning and prompt injection:** external content is stored as data with a trust label, wrapped in delimiters on retrieval, and the system prompt forbids following instructions found inside it.

- **Destructive actions** always need confirmation, even from learned workflows, unless the owner explicitly trusts that workflow.
- **Explainability:** each reply records the memory IDs it used so the dashboard can show why Ultron said or did something.
- **Forget controls:** hide an event (soft delete) or purge everything from one source.

13. Dashboard requirements

- Live conversation view with full-text search across all history.
- Facts table with approve, edit and reject, and fact history.
- Workflow manager: create, edit, set trust level, test run.
- Sources page: status, last sync, on/off, purge.
- Metrics: latency p50 and p95 per stage, command success rate, correction rate.
- Per-reply panel showing which memories were used.
- Export to markdown (for Obsidian) and as a training dataset.
- Read-only database connection; edits go through the memory service.

14. Evaluation

- **Command success rate** on a fixed test set of the owner's real commands, memory off versus on.
- **Correction rate over time** should fall.
- **Recall test:** about 30 questions on past conversations; track accuracy and the rate of invented memories.
- **Latency:** p50 and p95 per stage.
- **Persistence:** facts survive a restart, and a taught workflow works in a fresh session.

15. Later: training on the logged data

Curated data (verified successes; corrections as preference pairs) can be exported for LoRA fine-tuning with TRL, which Liquid's docs recommend for LFM models. Liquid's Home Assistant cookbook example (a small model fine-tuned to map sentences to tool calls, then merged and exported to GGUF) is a good template. Evaluate on a held-out set

before swapping models and keep the old model as rollback. Training on a 4 GB GPU is likely too tight, so decide in advance where training runs, because a cloud GPU means the data leaves the laptop. A first fine-tune could use synthetic data only.

16. Risks

Risk	Mitigation
Small model extracts wrong facts	Constrained JSON, validation rules, confidence thresholds, dashboard review queue, provenance
Poisoned memory or prompt injection from ingested content	Trust labels, delimiters, no instruction-following from sources, confirmation for risky actions
Latency creep as data grows	Budgeted retrieval, FTS indexes, RAM caches, precomputed embeddings, p95 monitoring
Stale fact used	Validity windows, recency and verification weighting, per-reply memory trace
Account bans from unofficial WhatsApp tools	Manual exports only
Gmail OAuth friction	Test-user mode, re-auth flow, verify Google policy early
Third-party privacy	Allow lists, local-only storage, purge controls
Data loss or corruption	WAL, nightly online backup, integrity check at startup
GPU contention on 4 GB VRAM	Background jobs only when idle; larger models on CPU

17. Delivery plan and open questions

Phase	Deliverable	Exit criteria
P0 Foundations	Fixed action schema with constrained output, cacheable prompt layout, metrics hooks	Commands parse reliably; static prefix cache hits confirmed in server logs
P1 Memory core	Database and migrations, single-writer queue, event logger, FTS5, RAM caches, alias and habit lookup, memory-aware prompt builder, read-only dashboard views	Every turn logged; survives restart; overhead within target
P2 Teach mode	Remember commands, workflows with executor and confirmations, corrections capture	A taught workflow runs correctly in a new session
P3 Learning loop	Idle consolidation, constrained-JSON extraction, summaries, decay, review queue	Facts appear with provenance; correction rate trends down
P4 Ingestion	Files watcher, WhatsApp export parser, YouTube transcripts, Gmail read-only sync, per-source controls	Ultron answers questions from each source and points to it
P5 Semantic recall	sqlite-vec embeddings and hybrid ranking, only if keyword search proves insufficient	Recall test improves with latency intact
P6 Training and screen learning	Dataset export, fine-tune trial, learn-from-screen research	Held-out evaluation beats the base model

The audio front end (noise suppression, speaker verification) is a separate workstream covered in the earlier research report.

Open questions for the engineering lead

1. Should the memory layer be an in-process module or a separate local service?
2. Which CPU-friendly embedding model should we benchmark for Phase 5?

3. Which model runs idle consolidation, given 4 GB VRAM?
4. Gmail OAuth mode and which labels are in scope.
5. Raw audio: never stored (recommended) or short expiring clips for debugging?
6. Strictly forget nothing, or allow a soft-delete “forget that” command?
7. Method for learning from the screen: accessibility tree or OCR?

Sources

- Mem0 repository
- Mem0: local coding agent with Ollama
- Letta memory docs
- Mem0 vs Letta comparison
- Zep paper (arXiv 2501.13956)
- Memory in the Age of AI Agents (survey)
- Voyager (arXiv 2305.16291)
- Agent Workflow Memory (arXiv 2409.07429)
- Cognee Gmail integration
- vstash paper
- TinyContext
- Liquid: llama.cpp chat and KV cache reuse
- llama.cpp discussion on prompt prefix caching
- Liquid: fine-tuning with TRL
- Liquid: Home Assistant fine-tuning example
- WhatsApp chat export guide
- WhatsApp unofficial libraries and ToS risk
- Obsidian Gmail sync (history cursor, desktop OAuth)
- google-readonly (testing-mode token expiry note)
- readvideo (transcript-first YouTube pipeline)
- summarize.sh YouTube mode
- LFM3-1.7B model card